

Skill-03

Code:

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <unistd.h>

#include <termios.h>


#define BUFFER_SIZE 100

#define MAX_COMMANDS 50


// Function to enable character-by-character input
void enableRawMode(struct termios *original) {
    struct termios raw;

    tcgetattr(STDIN_FILENO, original);

    raw = *original;

    // Disable canonical mode and echo
    raw.c_lflag &= ~(ICANON | ECHO);

    // Read one character at a time
    raw.c_cc[VMIN] = 1;
    raw.c_cc[VTIME] = 0;

    tcsetattr(STDIN_FILENO, TCSANOW, &raw);
}


// Function to restore normal terminal settings
void disableRawMode(struct termios *original) {
    tcsetattr(STDIN_FILENO, TCSANOW, original);
}
```

```

// Function to process commands
int processCommand(char *command) {

    // Exit condition
    if (strcmp(command, "exit") == 0 ||
        strcmp(command, "quit") == 0) {

        return 1;
    }

    // Help command
    if (strcmp(command, "help") == 0) {

        printf("\nAvailable commands:\n");
        printf(" help - Display available commands\n");
        printf(" hello - Display greeting\n");
        printf(" clear - Clear the screen\n");
        printf(" exit - Exit the program\n");
        printf(" quit - Exit the program\n");
    }

    // Hello command
    else if (strcmp(command, "hello") == 0) {

        printf("\nHello! Welcome to the interactive shell.\n");
    }

    // Clear command
    else if (strcmp(command, "clear") == 0) {

        printf("\033[2J\033[H");
    }

    // Empty input
    else if (strlen(command) == 0) {

```

```

        // Do nothing
    }

    // Unknown command
    else {

        printf("\nYou entered: %s\n", command);
    }

    return 0;
}

// Function to display all previously entered commands
void displayHistory(char history[][BUFFER_SIZE], int historyCount) {

    printf("\n===== \n");
    printf("    COMMAND HISTORY\n");
    printf("===== \n");

    if (historyCount == 0) {
        printf("No commands were entered.\n");
    }
    else {

        for (int i = 0; i < historyCount; i++) {

            printf("%d. %s\n", i + 1, history[i]);
        }
    }

    printf("===== \n");
}

int main() {

```

```
struct termios original;

// Buffer for the current command
char buffer[BUFFER_SIZE];

// Buffer to store all entered commands
char history[MAX_COMMANDS][BUFFER_SIZE];

// Number of commands entered
int historyCount = 0;

// Enable raw keyboard input
enableRawMode(&original);

// Main interactive loop
while (1) {

    int index = 0;
    char ch;

    // Clear current input buffer
    memset(buffer, 0, BUFFER_SIZE);

    // Display prompt
    printf("MyShell> ");
    fflush(stdout);

    // Read characters one by one
    while (1) {

        // Capture keyboard input
        if (read(STDIN_FILENO, &ch, 1) != 1) {
            continue;
        }
    }
}
```

```
// ENTER key

if (ch == '\n' || ch == '\r') {

    // Add string terminator

    buffer[index] = '\0';

    break;
}

// BACKSPACE key
if (ch == 127 || ch == 8) {

    if (index > 0) {

        index--;

        buffer[index] = '\0';

        // Remove character from screen

        printf("\b \b");

        fflush(stdout);

    }

    continue;
}

// CTRL+C
if (ch == 3) {

    printf("\n");

    disableRawMode(&original);

    return 0;
}
```

```

    }

    // Add normal character to input buffer
    if (index < BUFFER_SIZE - 1) {

        buffer[index] = ch;
        index++;

        // Display character on screen
        printf("%c", ch);
        fflush(stdout);
    }
}

/*
 * Store command in history.
 *
 * Do not store empty commands.
 * Do not store exit/quit because they are
 * used only to terminate the program.
 */
if (strlen(buffer) > 0 &&
    strcmp(buffer, "exit") != 0 &&
    strcmp(buffer, "quit") != 0) {

    if (historyCount < MAX_COMMANDS) {

        strcpy(history[historyCount], buffer);

        historyCount++;
    }
}

// Process the command
int shouldExit = processCommand(buffer);

```

```

// Exit condition

if (shouldExit) {

    // Display all previously entered commands

    displayHistory(history, historyCount);

    printf("\nGoodbye!\n");

    break;

}

printf("\n");

}

// Restore normal terminal settings

disableRawMode(&original);

return 0;

}

```

Output:

```

sohan_jadala@DESKTOP-OI7PGP2:~/2520080060_OSSP/2520080060_Skill$ gedit Skill2.c
** (gedit:1146): WARNING **: 06:54:32.571: Could not load Peas repository: Typelib file for namespace 'Peas', version '1.0' not found
** (gedit:1146): WARNING **: 06:54:32.571: Could not load PeasGtk repository: Typelib file for namespace 'PeasGtk', version '1.0' not found
sohan_jadala@DESKTOP-OI7PGP2:~/2520080060_OSSP/2520080060_Skill$ gcc Skill2.c -o Skill2
sohan_jadala@DESKTOP-OI7PGP2:~/2520080060_OSSP/2520080060_Skill$ ./Skill2
MyShell> hello
Hello! Welcome to the interactive shell.

MyShell> sohan jadala
You entered: sohan jadala

MyShell> AI&DS
You entered: AI&DS

MyShell> 2520080060
You entered: 2520080060

MyShell> Operating systems
You entered: Operating systems

MyShell> help
Available commands:
  help - Display available commands
  hello - Display greeting
  clear - Clear the screen
  exit - Exit the program
  quit - Exit the program

MyShell> exit
=====
COMMAND HISTORY
=====
1. hello
2. sohan jadala
3. AI&DS
4. 2520080060
5. Operating systems
6. help
=====
Goodbye!

```